

SAG Chronicles - Overall Architecture and Development Phases

Table of Contents

1. Context and Scope
2. C4 Architecture - Level 1: Context
3. C4 Architecture - Level 2: Container
4. C4 Architecture - Level 3: Component
5. C4 Architecture - Level 4: Code
6. Development Phases
7. Architectural Decisions

Context and Scope

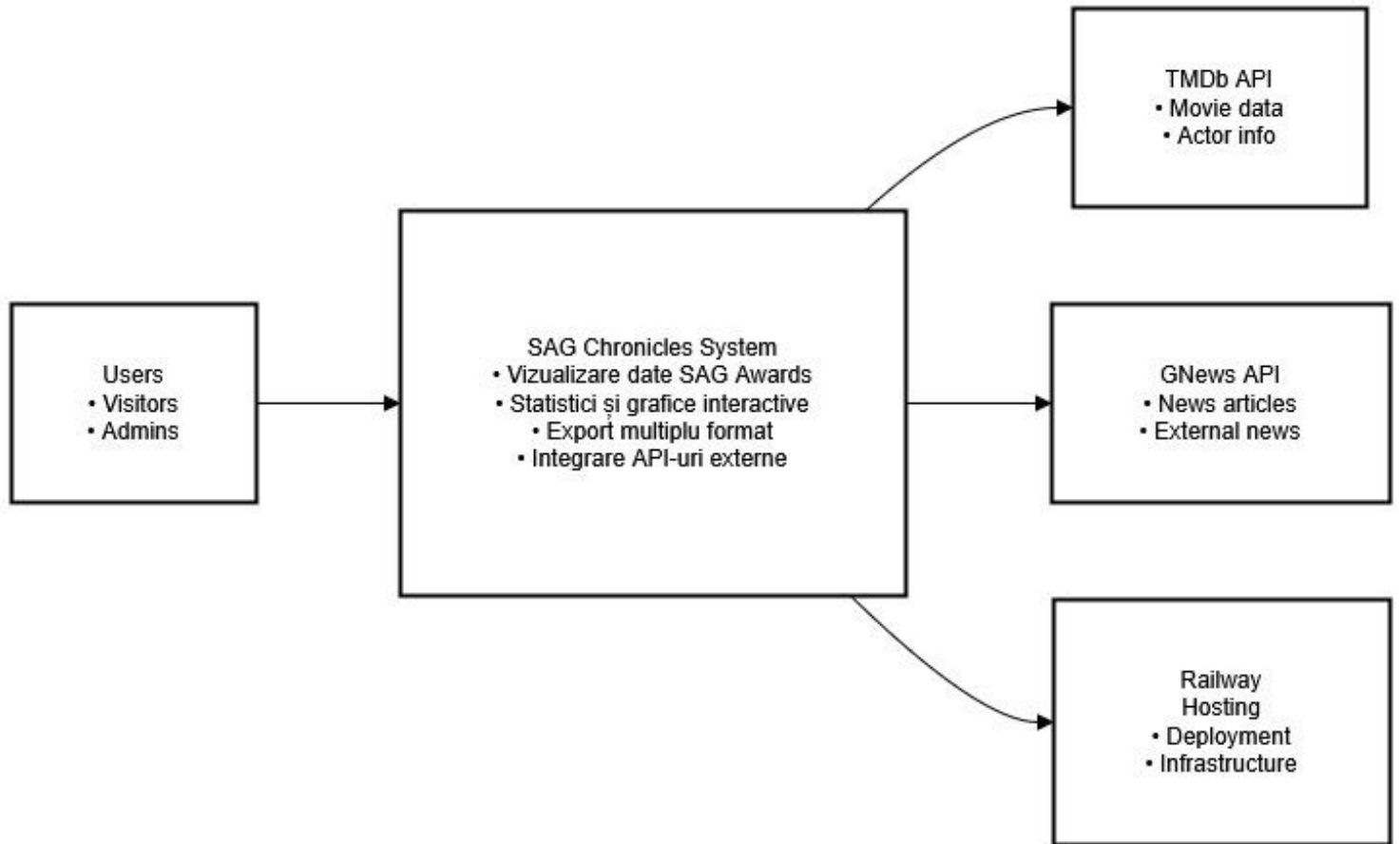
SAG Chronicles is a web application for flexible visualization of Screen Actors Guild (SAG) Awards nomination data from recent years. The application provides statistics, visualizations, and export functionality in multiple formats.

Main Objectives

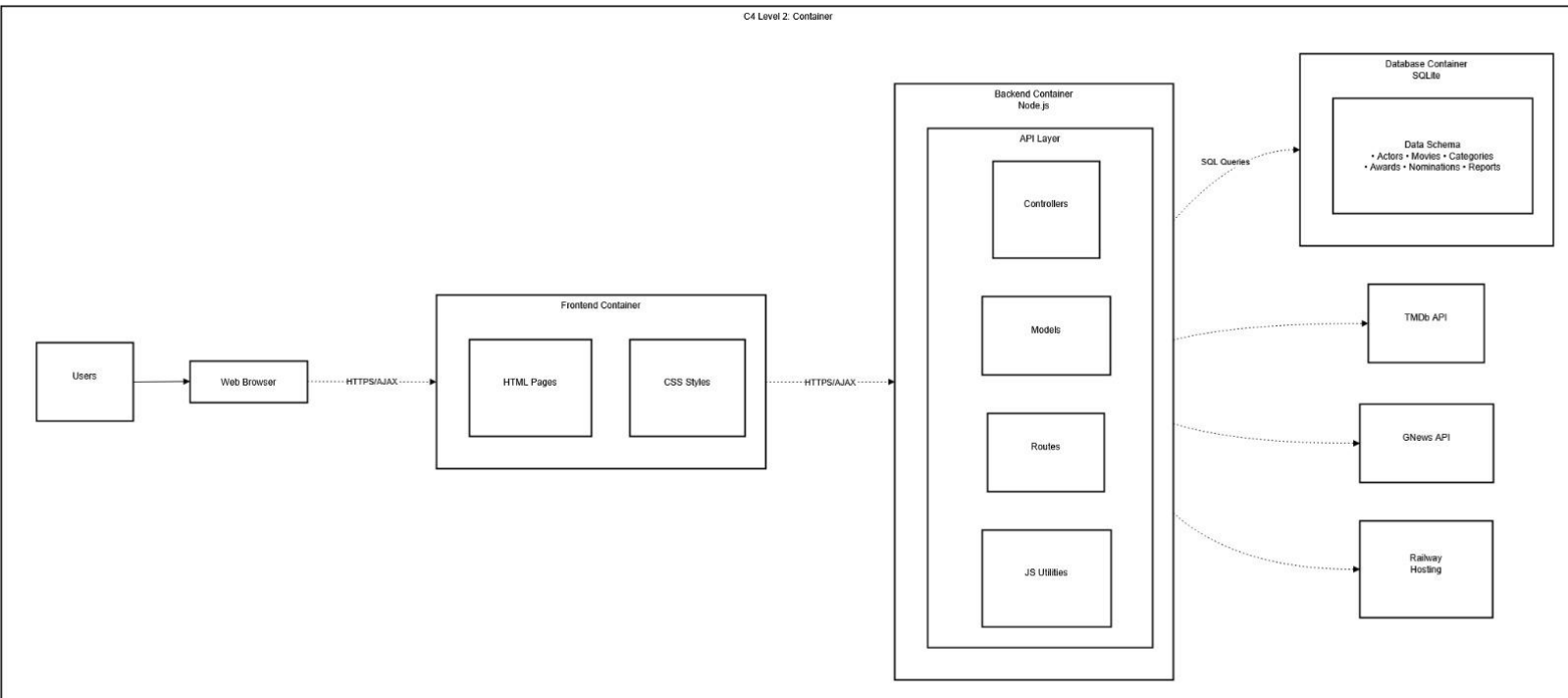
- Visualize SAG Awards data in at least 3 different ways
- Export in CSV, WebP, and SVG formats
- Integration with The Movie Database (TMDb) for additional information
- Support for configurable external news
- Responsive and modern interface

C4 Architecture - Level 1: Context

C4 Level 1: Context



C4 Architecture - Level 2: Container



C4 Architecture - Level 3: Component

Frontend Components

Frontend Container

Pages layer

- index.html (Landing page + navigation)
- search.html (Actor search + movies/news toggle)
- top-actors.html (Statistics visualization)
- category-stats.html (Category-based charts)
- winners-table.html (Winners data table)
- admin.html (Administrative interface)

Styling Layer

- responsive styles
- global.css
- admin.css
- category-stats.css
- fonts.css
- globals.css
- index.css
- search.css
- top-actors.css
- winners-table.css

Backend Components

Backend Container

Entry Point

- server.js

Routing Layer

- routes.js

Controllers Layer

- statsController.js
- searchController.js
- exportController.js
- adminController.js

Models Layer

- statsModel.js
- SearchMoviesModel.js

- GNewsModel.js
- adminModel.js

Data Layer

- setupDB.js
- importData.js
- actors_awards.db

JavaScript Utilities

- searchHelper.js
- topActorsHelper.js
- categoryStatsHelper.js
- winnersTableHelper.js
- exportHelper.js
- reportHelper.js

Configuration

- config.js
- package.json

C4 Architecture - Level 4: Code Example: Search Controller

javascript

```
// Controllers handle HTTP requests and coordinate between models class
SearchController {
  async handleCelebritySearch(req, res) {
    const query = req.query.query; try {
      // Coordinate with external API model const data = await
      SearchMoviesModel.fetchCelebrityNews(query);
      // Return structured response res.writeHead(200, { 'Content-Type':
      'application/json' }); res.end(JSON.stringify(data));
    } catch (error) {
      // Error handling res.writeHead(500, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ error: 'Server error' })); }
    }
  }
}
```

Example: Database Model

javascript

```
// Models handle data operations and business logic class StatsModel { static
getTopActorsByNominations(callback) {
  const query = `
    SELECT ACT.name, COUNT(N.id) as nomination_count
    FROM Nominations N
    JOIN Actors ACT ON N.actor_id = ACT.id
    JOIN Awards A ON N.award_id = A.id      WHERE A.year >= ?
    GROUP BY ACT.name
    ORDER BY nomination_count DESC
  `;
  db.all(query, [tenYearsAgo], callback); }
}
```

Development Phases

Phase 1: Foundation and Infrastructure

Estimated Duration: 1-2 weeks

Objectives

- Establish basic architecture
- Set up development environment
- Create project structure

Activities

1. Project Setup

- Initialize Git repository
- Configure package.json with necessary dependencies
- Establish folder structure

2. Server Configuration

- Implement server.js with Node.js HTTP server
- Create basic routing system
- Configure static file serving

3. Database Design

- Model data schema (Actors, Movies, Categories, Awards, Nominations)
- Implement setupDB.js for table creation
- Define relationships between entities

Phase 2: Backend Core and APIs

Estimated Duration: 2-3 weeks

Objectives

- Implement business logic

- Integrate with external APIs
- Create REST endpoints

Activities

1. Models and Database Operations

- statsModel.js - queries for statistics
- Implement CRUD operations

2. External API Integration

- SearchMoviesModel.js - TMDb integration
- GNewsModel.js - GNews integration
- Error handling

3. Controllers and Business Logic

- Implement statsController for statistics endpoints
- searchController for search functionality
- exportController for export functionalities

4. Data Import and Seed

- importData.js for initial database population
- Scripts for import from external sources

Phase 3: Frontend Development

Estimated Duration: 2-3 weeks

Objectives

- Create user interfaces
- Implement interactive functionalities
- Optimize for responsive design

Activities

1. HTML Structure and Layout

- index.html - landing page with navigation

- search.html - search interface
- top-actors.html - statistics page
- category-stats.html - category-based charts
- winners-table.html - winners table
- admin.html - administrative interface

2. **CSS Styling and Theming**

- globals.css - base styles and dark theme
- Implement responsive design
- Style individual components

3. **JavaScript Functionality**

- searchHelper.js - search logic and toggle movies/news
- Chart.js integration for visualizations

Phase 4: Advanced Features and Optimizations

Estimated Duration: 1-2 weeks

Objectives

- Implement advanced functionalities
- Testing and bug fixing

Activities

1. **Advanced Features**

- Admin panel (admin.html)
- Reporting system (reportHelper.js)
- Easter egg for admin access (7 clicks on logo)

2. **Export and Data Processing**

- exportHelper.js - export in multiple formats
- Data transformation and formatting

3. **Security and Validation**

- Input validation and sanitization
- SQL injection prevention
- XSS protection

Phase 5: Deployment and Finalization

Estimated Duration: 1 week

Objectives

- Prepare for production
- Deploy on Railway
- Final testing and documentation

Activities

1. **Deployment**

- Configure Railway hosting
- DNS setup

2. **Documentation and Testing**

- Finalize README.md
- User acceptance testing

Architectural Decisions

1. Technology Choices

Component	Technology	Justification
Backend	Node.js	Good learning experience, perfect for dynamic web pages
Database	SQLite	Simple setup, no complex configuration needed, great for small projects
Frontend	Vanilla JavaScript	Helps understand basics without frameworks
Charts	Chart.js	Easy to use library with good documentation
Hosting	Railway	Free to use, persistent database, relatively easy setup

2. Architectural Patterns

MVC Pattern (Modified)

- **Models:** Database operations and data handling
- **Views:** HTML templates and CSS styling
- **Controllers:** Request handling and response formatting

Service Layer Pattern

- External API calls encapsulated in services
- Logic reuse between controllers

Models (Data Layer)

- ├ Database operations (statsModel.js)
- ├ External API integration (SearchMoviesModel.js, GNewsModel.js)
- └ Business logic encapsulation

Views (Presentation Layer)

- ├ HTML templates (*.html)
- ├ CSS styling (styles/*.css)
- └ Client-side JavaScript (utils/*.js)

Controllers (Logic Layer)

- └ Request handling (controllers/*.js)
- └ Data transformation
- └ Response formatting

3. Security

Input Validation

javascript

```
// Example from searchController const query = req.query.query; if (!query || typeof query !== 'string' || query.length > 100) {  
  return res.status(400).json({ error: 'Invalid query parameter' }); }  
}
```

SQL Injection Prevention

javascript

```
// Prepared statements in all queries db.all(query, [sanitizedInput], callback);
```

XSS Protection

- Input sanitization in frontend
- Correct Content-Type headers
- Validation of data received from external APIs

5. Scalability

Horizontal Scaling

- CDN for static assets

Vertical Scaling

- Database query optimization
- Efficient data structures

Conclusions

SAG Chronicles implements a robust and scalable architecture that follows modern web development principles:

Strengths

- **Modularity:** Clear separation of responsibilities
- **Maintainability:** Well-organized and documented code
- **Performance:** Optimizations for speed and user experience
- **Security:** Preventive measures implemented

Modern Technologies

- REST API architecture
- Responsive web design
- Interactive data visualizations
- Multiple export formats
- External API integrations

Best Practices

- Separation of concerns
- DRY (Don't Repeat Yourself)
- Comprehensive error handling
- Input validation and sanitization